

EXP01

NSEW Intersection (+ shaped), All roads are of same size and length

State {number of cars} ; has ceiling value which is used to negatively penalize

Actions {0: NS green, 1: EW green} ; Mandatory yellow signal when changing actions, and when one lane(NS) is green automatically the other(EW) will be red and vice versa, hence so reducing the action space for convenience.

Transition: To simulate real-world scenario, each lane N,S,E,W has been assigned different probability of new incoming vehicle.

Reward function: Since the goal is to maintain traffic so that vehicles at each lane experiences least amount of waiting time, the model is penalized for the average waiting time of the vehicle in each lane, summed up as one single value.

Expected result: The model should assign more green time for the lane with higher vehicle incoming probability

Observation:

- Frequent signal switch
- The layout and state design is wrong, Each direction N,S,E,W should have distinct signals. (Existing design is wrong fundamentally w.r.t the traffic flow)
- Vehicle builds up in a single lane

Exp02

Everything same as 'Exp01' except,

Layout & design: Each Road(N,S,E,W) has distinct signals, only one can go green at a time

Reward function: $\alpha \cdot \text{Summation Average wait time of vehicle in each lane} + \beta \cdot \text{Switch penalty}$

$\alpha=1$; $\beta=10 \rightarrow$

Expected result: Only meaningful signal switch, real-life traffic simulation

Observation:

- Switches has been reduced significantly (but reward hacking)
- All signal turns yellow for any signal to turn green

- Agent keeps the other roads in red(waiting, till termination) even when it has to vehicle in current green road.(no meaningful signal switch)

Inference:

- Maybe since the feedback is summation of all roads, it doesn't know which road to free as clearing vehicle from any road will reduce the reward evenly. We can also observe that when trying to change the signal, green is being awarded for same signal subsequently, which also supports the claim.

Set different probability for vehicle entry in each road in each episode so that the agent learns to manage the crowd, if not it will learn for that specific setting.

The Project's aim is to demonstrate including the state of the consecutive signals into neighboring signals reward structure so that it is suspected to improve the performance of the traffic handling (based on waiting time). For this idea there is no need to demonstrate a large well managed traffic network, implementation on a straight road is enough which can be converted to signaled roads of any form. So focus on achieving the inclusion of neighboring signals state in reward structure, not on the implementation.

Exp03 ----- Enough for the project

Straight road with multiple signals (3), placed one after another; (S, A, R) same as before.

Environment: P -> Single prob.(dynamic) Of vehicle entering the lane, the vehicles continue through signals to exit the road

Observation:

- The amount of vehicle entry is static, but in real-life traffic magnitude may vary with time of the day. So, with the varying probability of vehicle entry if the amount of vehicle enter at each timestep also varies then the signal agent might learn to work on the number of vehicles waiting instead of adjusting itself to the static probability defined.

Exp04

Same as Exp03, only difference is reward function and environment

Environment: Dynamic magnitude of vehicle entry ($[2,8]$) at timestep 't' with probability 'p'

Now the magnitude of vehicle entry at each timestep is dynamic

Reward function : $n(n+1)/2$ where n is the number of vehicles in queue

When in green signal, the number of vehicles waiting will be reduced at a rate of 2 per timestep, then the reward will be calculated and will be replaced, not cumulated. So, this is like a zero-sum game where the vehicles are getting added and the agent tries to reduce the number of vehicles. The reward here is interpreted as the waiting time to leave(not finalized) and the penalty for the signal switch is also present.

Observation:

- The chokepoint is signal A (reason for maximum number of termination of episodes), Since the inflow of the vehicle is much greater than the outflow (because inflow is dynamic and outflow is static or small), even if the agent tries to keep the signal green, the vehicle amount increases which leads to the termination.
- In real world the incoming may or may not be this rigorous, sometime will be provided to reduce the traffic volume. The environment needs some adjustments

Exp05

Same as Exp04, only change is now the magnitude of vehicle exit at each timestep is dynamic.

Observation:

- At some episodes (eg: 285) the agent was able to keep the environment for up to 6000 steps , but subsequent episodes doesn't reach that high amount, but it reached more than 100, 200, or even 500 more frequently. But the log does not have this information because it records for every 10th episode, which misses these details.
- The agent hacked the reward. Kept all the signals green in that 445th episode, that is why the steps is so high.

- At (ep: 445) it went up and above 40000 steps, I had to manually terminate the program.

Inference:

- The agent is not positively rewarded for longer time of keeping the environment without terminating which. If we reward that behavior then it might make the agent motivate to keep the traffic running. I guess since this appraisal for good behavior is not there, the agent might settle for sub optimal behavior which is like (even how hard it works, it cannot get above certain level of reward) [This is the case before episode 445, the agent converged to the expected optimal behavior which is keeping all the signal green is achieved at episode 445]
- This is the exact behavior that is expected, since the straight road doesn't contain any intersections, it is common sense to keep the road open with only one signal in the starting. Now we have to proceed through adding intersections.

Exp 06

We add intersection to the signal 'B', now the shape of the road would be like a 'T'

Environment: Adding intersection at signal B; so now the signal B has to manage the hardest part. The probability of vehicle arrival and the new intersection is same as other roads. The vehicles coming out of intersection would join signal C. So, the straight road still stays as the single lane one way road.

Expected outcome: I suspect that the signal A and C would be always open and the B would regulate intersection by opening and closing.

Observation:

- At ep. 1500, the episodes still terminate due to ceiling amount of vehicles, the signals are not opening for roads which have higher number of vehicles, will wait for more episodes and record.
- Even after 3000 ep there was no improvement. Some signals does not care even if the amount of vehicles waiting reaches near the limit, why?

Inference:

- The reward is calculated separately for each and cumulated for training, this might be the reason, as even if the agent performs well and if another agent messes up, the good performer also gets punished for the good behavior and vice versa creating a whole confusion.

Exp07

Same setting as before, but now the agents are pure independent

Environment: We followed Centralized decision and Decentralized environment method before, now we switch to totally independent agent.

Expected result: If everything works well, signal A and C will only care about them and keep the signal always green which is the result we expected for the experiment before (exp06).

Observation:

- The signal A and B turned totally green and the B signal was switching, which was the expected behavior. Even if that's the case B didn't perform well as it was the reason for every termination of the episode because it didn't learn to handle the traffic, after A and C turned green totally. This is also supported as the negative reward value didn't decrease after certain level and episode.

Inference:

- But this behavior of signals (A and C) is not good. So this idea of independent agent is wrong, but this states that the other settings are tuned right, and the intersection has to be handled more effectively, and the model should be awarded for avoiding termination or a penalty for termination.
- CTDE works best. So improvement should be done on top of that.

Exp08----- From here on follow CTDE

Revert back to CTDE, Now we have to get the T intersection working right.

These are the current settings, up for modification.

$$S = [Q_A, Q_B, Q_C, Q_I, L_A, L_B, L_C, Y_A, Y_B, Y_C]$$

- **Queues** (Q_A, Q_B, Q_C, Q_I): The number of vehicles currently waiting at each signal (0 to 50). Q_I is the intersection road merging into B.
- **Light States** (L_A, L_B, L_C): * For A and C: 0 = Red, 1 = Green.
 - For B: 0 = Green for Main Road, 1 = Green for Intersection.
- **Yellow Timers** (Y_A, Y_B, Y_C): Countdown timers (3 steps) that lock the signal during a transition.

2. Action Space (A)

The agent uses a **Multi-Discrete** action space: $[2, 2, 2]$. At each step, it chooses:

- **Agent A:** Switch to Red (0) or Green (1).
- **Agent B:** Serve Main Road (0) or Serve Intersection (1).
- **Agent C:** Switch to Red (0) or Green (1).

Inflow

Vehicles arrive at Q_A and Q_I with probability p . If they arrive, a random number of cars (2 to 5) is added.

$$Q_{A,t+1} = Q_{A,t} + \text{Inflow} \quad \text{if } \text{rand}() < p$$

Outflow/Movement

Vehicles move only if the light state is Green ($L = 1$). The number of vehicles that *can* move is a random integer $V_{out} \in [2, 5]$.

- **A to B:** If $L_A = 1$, vehicles move from Q_A to Q_B .
- **B to C:** * If $L_B = 0$, vehicles move from Q_B to Q_C .
 - If $L_B = 1$, vehicles move from Q_I to Q_C .
- **C to Exit:** If $L_C = 1$, vehicles leave the system.

Yellow Logic

If an action at t differs from the action at $t - 1$, the light state becomes 2 (Yellow) and stays there for 3 steps before reaching the target state. No traffic moves during these 3 steps.

4. Reward Function (R)

This is the most critical part of the model's "brain." It is a **Cooperative Summed Reward**.

Individual Component:

For each queue, the penalty is calculated using an arithmetic series (triangular numbers):

$$r_i = -\frac{n(n+1)}{2}$$

Where n is the number of cars. This ensures that the penalty grows quadratically; having 10 cars in one lane is much worse than having 5 cars in two different lanes.

Switching Penalty:

Every time an agent initiates a state change (triggering a Yellow light), it receives an immediate penalty of -1.

Total Scalar Reward:

$$R_{total} = r_A + r_B + r_C + r_I + \sum \text{Switching Penalties}$$

PPO uses this total sum to update its policy.

Using same settings, but using DQN instead of PPO.

Observation

- No improvement, same state as Exp06

Exp 09:

Now will change the reward function

Reward: The model works on the aggregate reward, this does not tell which part made the problem so now I have to change the reward so that the good behavior of some agents does not get penalized for bad behavior of others.

Pressure based reward system:

Didn't work

Exp10

Implement using SUMO

Trained using PPO.

Worked best. Also managed traffic in most efficient way even when the inflow rate is increased(simultaneously giving chance for all roads in intersection)

Exp11

Implemented CTDE for baseline evaluation of the proposed method.

Proposed methods reward function:

Local Rewards:

- $r_{L1} = -(q_{E0} + q_{E1})$
- $r_{L2} = -(q_{E2} + q_{E3})$

The "Informed" Reward ($R_{proposed}$):

$$R_{Agent1} = (1 - \alpha)r_{L1} + \alpha(r_{L2})$$

$$R_{Agent2} = (1 - \alpha)r_{L2} + \alpha(r_{L1})$$

State

For Agent 1 (J_1):

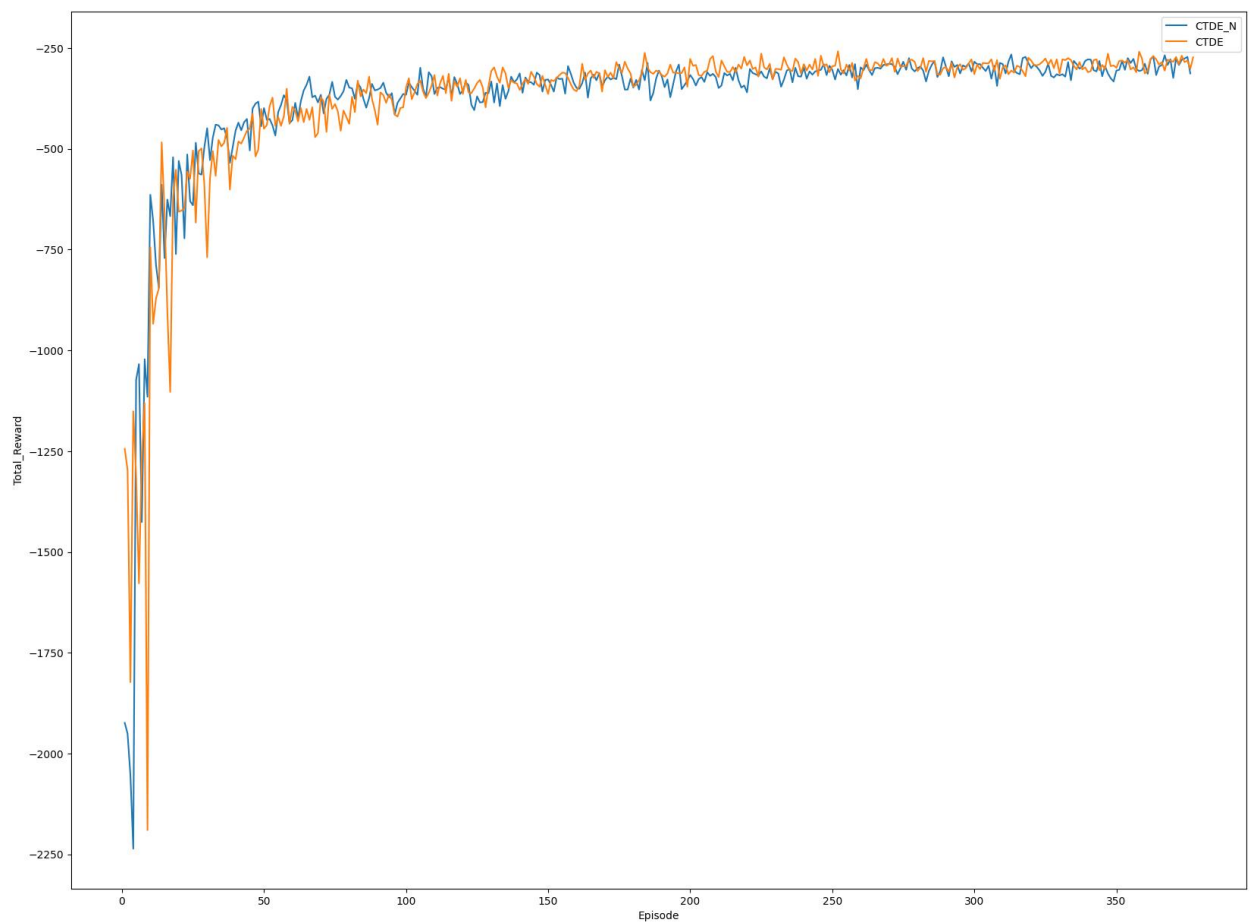
$$S_{J_1} = [q_{E0}, q_{E1}, \phi_{J_1}, q_{E3}, \phi_{J_2}]$$

- **Local:** Queues at West ($E0$) and South ($E1$), plus its own phase (ϕ_{J_1}).
- **Neighbor (Informed):** It now sees the North queue ($E3$) and what the neighbor is currently doing (ϕ_{J_2}).

For Agent 2 (J_2):

$$S_{J_2} = [q_{E2}, q_{E3}, \phi_{J_2}, q_{E0}, \phi_{J_1}]$$

Observation:



Suspect:

- The step size is decreasing as episode goes, which is not good. If the agent actually learned, it would've increased. Your agent is likely "failing faster." If the reward is the same but the episodes are shorter, the agent has found a way to reach the "Gridlock" state (Total Halting > 150) quickly.
- Have to implement switch penalty to stop frequent switches.
- Include termination and huge penalty for termination

Variance of df Total_Reward: 45230.36195744678

Variance of df Total_Reward: -378.47340425531917

Variance of df Steps: 7.007141843971664

Variance of df1 Total_Reward: 41846.05200632089

Variance of df1 Total_Reward: -376.6684350132626

Variance of df1 Steps: 7.132541339804619

EXP12

Implemented what has been suspected in last experiment and with that

- Yellow light implementation and switch penalty
- Increasing the window for termination (more cars > 300 can stand in a queue)
- Different inflow in different roads

Exp0

Changes to vehicle inflow in the intersection road.

Environment: With probability q the vehicles from the intersection road can go either to left (to signal C) or right (to signal A)